

Phase 3: Searching in Array ADT

1. Big Idea of Phase 3

Phase 3 teaches how to **search for a value inside an Array ADT**.

In Phase 1, you learned how an Array ADT stores data:

```
struct Array {  
    int A[20];  
    int size;  
    int length;  
};
```

In Phase 2, you learned how to display, append, insert, and delete values.

In Phase 3, you now learn how to answer this question:

Is a particular value present in the array? If yes, at which index?

The main topics in Phase 3 are:

1. Linear search
2. Successful and unsuccessful search
3. Best, worst, unsuccessful, and average time complexity
4. Improving linear search
5. Transposition
6. Move-to-front
7. Swap helper function
8. Pass-by-value vs pass-by-address for search functions

Important correction:

Phase 3 is about **linear search** and improved linear search. Binary search comes in Phase 4.

2. Quick Review: Why `length` Matters

Before searching, remember this rule:

```
size    = total capacity
length = number of valid elements
```

Example:

```
struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};
```

Memory view:

Index	0	1	2	3	4	5	6	...	19
Value	2	3	4	5	6	-	-	...	-

Here:

```
size = 20
length = 5
```

That means valid elements are only from:

```
index 0 to index 4
```

So search must check only:

```
A[0], A[1], A[2], A[3], A[4]
```

It should **not** check all the way to `size`, because the unused slots are not real logical data.

Main rule:

Searching must loop from index `0` to `length - 1`.

3. What Does Searching Mean?

Searching means checking whether a value exists inside the array.

The value we are looking for is called the **key**.

Example:

```
A = [8, 9, 4, 7, 6, 3, 10, 5, 14, 2]
key = 5
```

Question:

Where is 5?

Answer:

5 is at index 7

So the search function should return:

7

4. Key

The **key** is the value we want to find.

Example:

```
key = 5
```

This means:

Search the array and find where the value 5 is stored.

The key is not an index. It is the value being searched.

Key vs Index

Term	Meaning	Example
Key	The value we are looking for	5
Index	The position where the value is found	7

Example:

```
A = [8, 9, 4, 7, 6, 3, 10, 5]
```

Here:

```
key = 5  
index = 7
```

Because `5` is stored at `A[7]`.

5. Successful Search

A search is **successful** when the key exists in the array.

Example:

```
A = [2, 3, 4, 5, 6]  
key = 5
```

The value `5` exists at index `3`.

So the search returns:

```
3
```

Meaning:

The key was found at index `3`.

6. Unsuccessful Search

A search is **unsuccessful** when the key does not exist in the array.

Example:

```
A = [2, 3, 4, 5, 6]  
key = 10
```

The value `10` is not in the array.

So the search returns:

-1

Why return -1 ?

Because valid array indexes are:

0, 1, 2, ..., length - 1

So -1 is never a valid index.

That makes -1 a clear signal meaning:

Not found.

7. Linear Search

7.1 What Linear Search Means

Linear search checks the array **one element at a time from left to right**.

It starts at index 0 and keeps moving forward until one of two things happens:

1. The key is found.
2. The end of the valid array is reached.

Example:

A = [8, 9, 4, 7, 6, 3, 10, 5]
key = 5

Linear search checks:

A[0] = 8 not found
A[1] = 9 not found
A[2] = 4 not found
A[3] = 7 not found
A[4] = 6 not found
A[5] = 3 not found

```
A[6] = 10  not found
A[7] = 5   found
```

Return:

7

It is called **linear** because it moves in a straight line through the array.

8. Linear Search Code

There are two common ways to write linear search in this topic.

One version receives the structure by value because it only reads the array:

```
int linearSearch(struct Array arr, int key) {
    for (int i = 0; i < arr.length; i++) {
        if (arr.A[i] == key) {
            return i;
        }
    }

    return -1;
}
```

Another version receives a pointer:

```
int linearSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            return i;
        }
    }

    return -1;
}
```

Both versions can work.

For beginner learning:

- Use **pass by value** when the function only reads the array.

- Use **pass by address** when the function changes the array.

Normal linear search only reads, so pass by value is enough.

Improved search changes the array order, so it needs pass by address.

9. Explanation of Linear Search Code

Code:

```
for (int i = 0; i < arr.length; i++)
```

Meaning:

Start from index `0` and continue while `i` is less than `length`.

This makes the loop visit only valid elements.

Important:

```
arr.length
```

should be used, not:

```
arr.size
```

Because `size` includes unused capacity.

Code:

```
if (arr.A[i] == key)
```

Meaning:

Check whether the current element is equal to the key.

Code:

```
return i;
```

Meaning:

If the key is found, immediately return its index.

The function stops as soon as it finds the key.

Code:

```
return -1;
```

Meaning:

If the loop finishes and no match was found, return `-1`.

This must be written **after the loop**, not inside the loop.

Wrong idea:

```
for (int i = 0; i < arr.length; i++) {  
    if (arr.A[i] == key) {  
        return i;  
    } else {  
        return -1;    // wrong  
    }  
}
```

Why is this wrong?

Because it returns `-1` after checking only the first element.

Correct idea:

```
for (int i = 0; i < arr.length; i++) {  
    if (arr.A[i] == key) {  
        return i;  
    }  
}  
return -1;
```


This checks all valid elements before deciding the key is missing.

10. Successful Linear Search Dry Run

Given:

```
A = [8, 9, 4, 7, 6, 3, 10, 5, 14, 2]
length = 10
key = 5
```

Trace:

Step	i	A[i]	Match?	Action
1	0	8	No	Move to next index
2	1	9	No	Move to next index
3	2	4	No	Move to next index
4	3	7	No	Move to next index
5	4	6	No	Move to next index
6	5	3	No	Move to next index
7	6	10	No	Move to next index
8	7	5	Yes	Return 7

Result:

```
return 7;
```

Meaning:

The key 5 was found at index 7.

11. Unsuccessful Linear Search Dry Run

Given:

```
A = [8, 9, 4, 7, 6, 3, 10, 5, 14, 2]
length = 10
key = 12
```

Trace:

Step	i	A[i]	Match?	Action
1	0	8	No	Move to next index
2	1	9	No	Move to next index
3	2	4	No	Move to next index
4	3	7	No	Move to next index
5	4	6	No	Move to next index
6	5	3	No	Move to next index
7	6	10	No	Move to next index
8	7	5	No	Move to next index
9	8	14	No	Move to next index
10	9	2	No	Move to next index
11	10	N/A	Loop stops	Return -1

Result:

```
return -1;
```

Meaning:

The key 12 is not present in the array.

12. Linear Search and Duplicates

If the array has duplicate values, normal linear search returns the **first matching index**.

Example:

```
A = [4, 8, 6, 8, 10]
key = 8
```

There are two 8 s:

```
A[1] = 8
A[3] = 8
```

Linear search starts from the left.

So it finds the first 8 at index 1 and returns:

1

It does not continue to find the second 8.

Important note:

A basic linear search returns the first match, not all matches.

13. Complexity of Linear Search

Let:

```
N = length
```

N means the number of valid elements.

Do not use size for complexity here, because search only checks valid elements from 0 to length - 1.

13.1 Best Case

Best case means the key is found immediately at index 0.

Example:

```
A = [5, 8, 2, 9]
key = 5
```

Only one comparison is needed:

```
A[0] == 5
```

Time complexity:

```
O(1)
```

Reason:

The search finishes after checking the first element.

13.2 Worst Case

Worst case means the key is at the last valid index.

Example:

```
A = [8, 2, 9, 5]
key = 5
```

The search checks:

```
8, 2, 9, 5
```

It checks all **N** elements before finding the key.

Time complexity:

```
O(N)
```

Reason:

The search may need to scan the whole array.

13.3 Unsuccessful Case

Unsuccessful case means the key is missing.

Example:

```
A = [8, 2, 9, 5]
key = 10
```

The search must check every valid element before it can say:

```
not found
```

Time complexity:

```
O(N)
```

Reason:

To be sure the key is missing, every valid element must be checked.

13.4 Average Case

Average case means the key may be found somewhere in the middle.

If the key is equally likely to be at any position, the number of comparisons may be:

```
1, 2, 3, ..., N
```

Total comparisons:

```
1 + 2 + 3 + ... + N
```

Formula:

```
N(N + 1) / 2
```

Average comparisons:

$$[N(N + 1) / 2] / N$$

Simplified:

$$(N + 1) / 2$$

Since this grows linearly as N grows, the average time complexity is:

$$O(N)$$

Important:

Even though the exact average is $(N + 1) / 2$, Big O ignores constants, so the average case is $O(N)$.

14. Linear Search Complexity Summary

Case	Situation	Time Complexity
Best case	Key is at index 0	$O(1)$
Worst case	Key is at the last index	$O(N)$
Unsuccessful case	Key is missing	$O(N)$
Average case	Key is somewhere in the middle	$O(N)$
Space complexity	Only uses a few variables	$O(1)$

15. Why Improve Linear Search?

Normal linear search is simple, but it has a weakness:

If the same value is searched again and again, linear search may repeatedly do the same work.

Example:

```
A = [2, 3, 4, 5, 6]
key = 5
```

Normal linear search checks:

```
2, 3, 4, 5
```

If you search for `5` again, it again checks:

```
2, 3, 4, 5
```

That is repetitive.

So we can improve repeated searching by moving frequently searched values closer to the front.

Why?

Because values near the front are found faster.

Phase 3 uses two improvement methods:

1. Transposition
2. Move-to-front

16. Swap Helper Function

Before learning the improved search methods, we need a function to swap two values.

Code:

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

This function receives addresses, not normal values.

That is why the parameters are:

```
int *x  
int *y
```

They point to actual memory locations.

17. Why Swap Needs `temp`

Suppose we want to swap these two values:

```
x = 5  
y = 4
```

Wrong idea:

```
*x = *y;  
*y = *x;
```

After the first line:

```
*x = 4
```

The original `5` is lost.

So we need a temporary variable:

```
int temp = *x;  
*x = *y;  
*y = temp;
```

Step-by-step:

Step	Action	Result
1	Save first value	<code>temp = 5</code>
2	Copy second value into first	<code>x = 4</code>
3	Copy saved value into second	<code>y = 5</code>

Final result:


```
x = 4
y = 5
```

Memory trick:

`temp` protects the first value from being lost.

18. Why Improved Search Uses Pass-by-Address

Normal linear search only reads the array.

So it can receive the structure by value:

```
int linearSearch(struct Array arr, int key)
```

But improved search changes the array by swapping elements.

So it must receive the actual structure address:

```
int transpositionSearch(struct Array *arr, int key)
int moveToFrontSearch(struct Array *arr, int key)
```

Because `arr` is a pointer, we use the arrow operator:

```
arr->length
arr->A[i]
```

Not the dot operator:

```
arr.length    // wrong when arr is a pointer
arr.A[i]      // wrong when arr is a pointer
```

Summary:

Function	Changes array?	Parameter style
Normal linear search	No	By value is enough
Transposition search	Yes	By address

Function	Changes array?	Parameter style
Move-to-front search	Yes	By address

19. Improved Search Method 1: Transposition

19.1 What Transposition Means

Transposition means:

When the key is found, swap it with the element just before it.

So the found value moves one step closer to the front.

Example:

```
A = [2, 3, 4, 5, 6]
key = 5
```

The key **5** is at index **3**.

The value before it is **4** at index **2**.

Swap them:

```
[2, 3, 4, 5, 6]
```

becomes:

```
[2, 3, 5, 4, 6]
```

Now **5** is closer to the front.

If we search for **5** again, it will be found faster than before.

20. Transposition Search Code

```
int transpositionSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
```

```

        if (arr->A[i] == key) {
            if (i > 0) {
                swap(&arr->A[i], &arr->A[i - 1]);
                return i - 1;
            }

            return i;
        }
    }

    return -1;
}

```

21. Explanation of Transposition Code

Code:

```
for (int i = 0; i < arr->length; i++)
```

Meaning:

Search from left to right through the valid elements.

Code:

```
if (arr->A[i] == key)
```

Meaning:

If the key is found, prepare to move it closer to the front.

Code:

```
if (i > 0)
```

Meaning:

Only swap with the previous element if the key is not already at the front.

Why is this needed?

If `i = 0`, then `i - 1 = -1`.

This would be invalid:

```
arr->A[-1]
```

So we must protect the code with:

```
if (i > 0)
```

Code:

```
swap(&arr->A[i], &arr->A[i - 1]);
```

Meaning:

Swap the found element with the previous element.

The `&` symbol sends the address of each element to the swap function.

Code:

```
return i - 1;
```

Meaning:

Return the new index of the key after swapping.

If the key was at index `3`, after swapping with index `2`, the key is now at index `2`.

So return:

```
2
```

22. Transposition Dry Run

Array:

```
A = [2, 3, 4, 5, 6]
key = 5
```

Trace:

Step	i	A[i]	Match?	Action	Array
1	0	2	No	Move next	[2, 3, 4, 5, 6]
2	1	3	No	Move next	[2, 3, 4, 5, 6]
3	2	4	No	Move next	[2, 3, 4, 5, 6]
4	3	5	Yes	Swap A[3] and A[2]	[2, 3, 5, 4, 6]

Before swap:

```
[2, 3, 4, 5, 6]
```

Swap:

```
A[3] with A[2]
```

After swap:

```
[2, 3, 5, 4, 6]
```

Return:

```
2
```

Why 2?

Because after the swap, the key 5 is now at index 2.

23. Repeated Transposition Example

Start:

```
A = [2, 3, 4, 5, 6]  
key = 5
```

First search:

```
[2, 3, 4, 5, 6]
```

5 is found at index 3, then swapped with index 2:

```
[2, 3, 5, 4, 6]
```

Second search:

5 is found at index 2, then swapped with index 1:

```
[2, 5, 3, 4, 6]
```

Third search:

5 is found at index 1, then swapped with index 0:

```
[5, 2, 3, 4, 6]
```

Now future searches for 5 are very fast.

Important idea:

Transposition improves repeated searches gradually.

It does not move the value to the front immediately.

24. Improved Search Method 2: Move-to-Front

24.1 What Move-to-Front Means

Move-to-front means:

When the key is found, swap it directly with the first element.

Example:

```
A = [2, 3, 4, 5, 6]
key = 5
```

The key **5** is at index **3**.

Move-to-front swaps **A[3]** with **A[0]**.

Before:

```
[2, 3, 4, 5, 6]
```

After:

```
[5, 3, 4, 2, 6]
```

Now the key is at the front.

If we search for **5** again, it is found immediately at index **0**.

25. Move-to-Front Search Code

```
int moveToFrontSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            swap(&arr->A[i], &arr->A[0]);
            return 0;
        }
    }
}
```

```
    return -1;  
}
```

26. Explanation of Move-to-Front Code

Code:

```
for (int i = 0; i < arr->length; i++)
```

Meaning:

Search valid elements from left to right.

Code:

```
if (arr->A[i] == key)
```

Meaning:

If the key is found, move it to the front.

Code:

```
swap(&arr->A[i], &arr->A[0]);
```

Meaning:

Swap the found element with the first element.

Code:

```
return 0;
```

Meaning:

After the swap, the key is now at index `0`, so return `0`.

Code:

```
return -1;
```

Meaning:

If the key is not found after checking all valid elements, return `-1`.

27. Move-to-Front Dry Run

Array:

```
A = [2, 3, 4, 5, 6]
key = 5
```

Trace:

Step	i	A[i]	Match?	Action	Array
1	0	2	No	Move next	[2, 3, 4, 5, 6]
2	1	3	No	Move next	[2, 3, 4, 5, 6]
3	2	4	No	Move next	[2, 3, 4, 5, 6]
4	3	5	Yes	Swap A[3] and A[0]	[5, 3, 4, 2, 6]

Before swap:

```
[2, 3, 4, 5, 6]
```

Swap:

```
A[3] with A[0]
```

After swap:

```
[5, 3, 4, 2, 6]
```

Return:

0

Why 0?

Because after swapping, the key 5 is at index 0.

28. Repeated Move-to-Front Example

Start:

A = [2, 3, 4, 5, 6]
key = 5

First search:

The key 5 is found at index 3.

Move it to the front:

[5, 3, 4, 2, 6]

Second search:

The key 5 is now at index 0.

So it is found immediately.

Time for second search:

$O(1)$

Important idea:

Move-to-front improves repeated search immediately.

29. Transposition vs Move-to-Front

Feature	Transposition	Move-to-front
Movement	Moves key one step forward	Moves key directly to index <code>0</code>
Speed of improvement	Gradual	Immediate
Array order change	Smaller change	Bigger change
Return value after match	New index, usually <code>i - 1</code>	Always <code>0</code>
Good for	Mild reordering	Frequently repeated searches

30. When to Use Transposition

Use transposition when:

- You want to improve repeated searches slowly.
- You do not want to disturb the array order too much.
- You want frequently used values to gradually move forward.

Example:

```
[2, 3, 4, 5, 6]
```

After searching `5` once:

```
[2, 3, 5, 4, 6]
```

Only a small change happened.

31. When to Use Move-to-Front

Use move-to-front when:

- The same values are searched many times.
- You want fast repeated searches.
- You are okay with changing the array order more strongly.

Example:

```
[2, 3, 4, 5, 6]
```

After searching **5** once:

```
[5, 3, 4, 2, 6]
```

The searched value jumps directly to the front.

32. Important Warning: Sorted Arrays

Do not use transposition or move-to-front if the array must stay sorted.

Why?

Because both methods change the order of elements.

Example sorted array:

```
[2, 3, 4, 5, 6]
```

Move-to-front search for **5** gives:

```
[5, 3, 4, 2, 6]
```

This is no longer sorted.

Important:

Improved linear search is useful for repeated searching, but it can destroy sorted order.

This matters because binary search, which comes in Phase 4, requires a sorted array.

33. Complete Phase 3 Program

```
#include <stdio.h>

struct Array {
    int A[20];
```

```

    int size;
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int linearSearch(struct Array arr, int key) {
    for (int i = 0; i < arr.length; i++) {
        if (arr.A[i] == key) {
            return i;
        }
    }

    return -1;
}

int transpositionSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            if (i > 0) {
                swap(&arr->A[i], &arr->A[i - 1]);
                return i - 1;
            }

            return i;
        }
    }

    return -1;
}

int moveToFrontSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            swap(&arr->A[i], &arr->A[0]);
            return 0;
        }
    }
}

```

```

    }

    return -1;
}

int main() {
    struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};

    printf("Original array:\n");
    display(arr);

    int index = linearSearch(arr, 5);
    printf("Linear search result for 5: %d\n", index);

    index = transpositionSearch(&arr, 5);
    printf("After transposition search for 5, index: %d\n", index);
    display(arr);

    index = moveToFrontSearch(&arr, 5);
    printf("After move-to-front search for 5, index: %d\n", index);
    display(arr);

    return 0;
}

```

34. Program Walkthrough

Starting array:

[2, 3, 4, 5, 6]

Step 1: Display Original Array

Output:

2 3 4 5 6

Step 2: Normal Linear Search for 5

Call:

```
linearSearch(arr, 5);
```

The function checks:

```
2, 3, 4, 5
```

It finds **5** at index **3**.

Return:

```
3
```

Array stays the same:

```
[2, 3, 4, 5, 6]
```

Why?

Because normal linear search only reads the array.

Step 3: Transposition Search for **5**

Call:

```
transpositionSearch(&arr, 5);
```

The key **5** is found at index **3**.

Swap it with the previous element at index **2**.

Before:

```
[2, 3, 4, 5, 6]
```

After:

[2, 3, 5, 4, 6]

Return:

2

Why?

Because 5 moved from index 3 to index 2.

Step 4: Move-to-Front Search for 5

Current array after transposition:

[2, 3, 5, 4, 6]

Call:

```
moveToFrontSearch(&arr, 5);
```

The key 5 is found at index 2.

Swap it with index 0.

Before:

[2, 3, 5, 4, 6]

After:

[5, 3, 2, 4, 6]

Return:

0

Why?

Because 5 is now at the front.

35. Possible Output

```
Original array:
2 3 4 5 6
Linear search result for 5: 3
After transposition search for 5, index: 2
2 3 5 4 6
After move-to-front search for 5, index: 0
5 3 2 4 6
```

Important:

The final array depends on which improved searches you run and in what order.

Why?

Because improved searches modify the array.

36. Operation Summary Table

Operation	Meaning	Changes array?	Return value	Time Complexity
Linear search	Search from left to right	No	Index or -1	Best $O(1)$, worst $O(N)$
Transposition search	Search, then move found value one step forward	Yes	New index or -1	First search $O(N)$, repeated searches improve gradually
Move-to-front search	Search, then move found value to index 0	Yes	0 or -1	First search $O(N)$, immediate repeated search $O(1)$

37. Common Beginner Mistakes in Phase 3

Mistake 1: Searching up to size

Wrong:

```
for (int i = 0; i < arr.size; i++)
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Reason:

Search only valid elements, not unused capacity.

Mistake 2: Returning `-1` too early

Wrong:

```
for (int i = 0; i < arr.length; i++) {  
    if (arr.A[i] == key) {  
        return i;  
    } else {  
        return -1;  
    }  
}
```

Correct:

```
for (int i = 0; i < arr.length; i++) {  
    if (arr.A[i] == key) {  
        return i;  
    }  
}  
return -1;
```

Reason:

Return `-1` only after checking all valid elements.

Mistake 3: Thinking `-1` is an error

`-1` is not a programming error here.

It is a planned return value that means:

not found

Mistake 4: Forgetting that duplicates return the first match

Example:

```
A = [4, 8, 6, 8, 10]
key = 8
```

Linear search returns:

1

It does not return index 3, because it stops at the first match.

Mistake 5: Using dot instead of arrow in pointer functions

Wrong:

```
arr.length
arr.A[i]
```

Correct inside pointer functions:

```
arr->length
arr->A[i]
```

Reason:

If the parameter is `struct Array *arr`, use `->`.

Mistake 6: Forgetting `i > 0` in transposition

Wrong:

```
swap(&arr->A[i], &arr->A[i - 1]);
```

If `i = 0`, this tries to access:

```
A[-1]
```

Correct:

```
if (i > 0) {  
    swap(&arr->A[i], &arr->A[i - 1]);  
}
```

Mistake 7: Using improved search on a sorted array that must stay sorted

Example:

```
[2, 3, 4, 5, 6]
```

Move-to-front search for `5` gives:

```
[5, 3, 4, 2, 6]
```

Now the array is not sorted.

This can cause problems later for binary search.

38. Beginner Memory Tricks

For Linear Search

Linear search = check one by one from left to right.

For Key

Key = the value you are trying to find.

For Successful Search

Found? Return the index.

For Unsuccessful Search

Not found? Return `-1`.

For Transposition

Found value moves one step forward.

For Move-to-Front

Found value jumps to index `0`.

For Improved Search

Faster repeated searches, but array order changes.

39. Phase 3 Checkpoint Questions

You understand Phase 3 when you can answer these without looking:

1. What does searching mean?
2. What is a key?
3. What does successful search return?
4. What does unsuccessful search return?
5. Why is `-1` used for unsuccessful search?
6. What does linear search do?
7. Why does linear search start at index `0`?
8. Why should linear search use `length`, not `size`?
9. When does linear search stop?
10. What happens if the array has duplicates?
11. What is the best-case time complexity of linear search?
12. What is the worst-case time complexity of linear search?
13. What is the unsuccessful-case time complexity of linear search?
14. Why is the average case also `O(N)`?
15. Why do we improve linear search?
16. What does the swap function do?
17. Why does swap need a temporary variable?
18. Why does improved search use pass-by-address?
19. What does transposition do?
20. Why does transposition check `i > 0`?
21. What does move-to-front do?

- 22. Why does move-to-front return 0 after a successful match?
- 23. Which method changes the array more: transposition or move-to-front?
- 24. Why should improved search not be used when sorted order must be preserved?
- 25. What is the main difference between Phase 3 and Phase 4?

40. Phase 3 Final Summary

Phase 3 teaches searching in an Array ADT.

The main search method is **linear search**.

Linear search checks valid elements from left to right:

A[0], A[1], A[2], ..., A[length - 1]

If the key is found, it returns the index.

If the key is missing, it returns:

-1

Linear search complexity:

Case	Time
Best	$O(1)$
Worst	$O(N)$
Unsuccessful	$O(N)$
Average	$O(N)$
Space	$O(1)$

Phase 3 also teaches two ways to improve repeated linear search:

1. **Transposition**: move the found value one step forward.
2. **Move-to-front**: move the found value directly to index 0.

These improved methods can make repeated searches faster, but they change the array order.

So they should not be used when the array must stay sorted.

41. One-Line Final Memory

Phase 3 is about searching valid elements from left to right, returning the index if found, returning `-1` if missing, and using transposition or move-to-front to make repeated searches faster.